

98. Validate Binary Search Tree

```
class Solution {  
    void inorder(TreeNode root,ArrayList<Integer> al){  
        if(root==null) return;  
        inorder(root.left,al);  
        al.add(root.val);  
        inorder(root.right,al);  
    }  
    public boolean isValidBST(TreeNode root) {  
        ArrayList<Integer> al=new ArrayList<>();  
        inorder(root,al);  
        for(int i=0;i<al.size()-1;i++){  
            if(al.get(i)>=al.get(i+1)){  
                return false;  
            }  
        }  
        return true;  
    }  
}
```

230. Kth Smallest Element in a BST

```
class Solution {  
    void preorder(TreeNode root,List<Integer> al){  
        if(root==null){  
            return;  
        }  
        al.add(root.val);  
    }  
}
```

```

        preorder(root.left,al);
        preorder(root.right,al);
    }

    public int kthSmallest(TreeNode root, int k) {
        List<Integer> al=new ArrayList<>();
        preorder(root,al);
        Collections.sort(al);
        return al.get(k-1);
    }
}

```

110. Balanced Binary Tree

```

class Solution {
    boolean flag=true;

    public int height(TreeNode root){
        if(root==null){
            return 0;
        }
        int leftHeight=height(root.left);
        int rightHeight=height(root.right);
        if(Math.abs(leftHeight-rightHeight)>1){
            flag=false;
        }
        return 1+Math.max(leftHeight,rightHeight);
    }

    public boolean isBalanced(TreeNode root) {
        height(root);
        return flag;
    }
}

```

